Photo by TheOtherKev on pixabay

Lecture 5

–

Optimal Classifier
Gaussian Classifier
Naïve Bayes Classifier

Schachbrett (*Melanargia galathea*)

- Das Schachbrett ist eine sehr weit verbreitete und variable Art und bildete eine Anzahl örtlicher Formen aus. Der Falter ist Schmetterling des Jahres 2019.
- Die Falter erreichen eine Flügelspannweite von 37 bis 52 Millimetern. Ihre Flügeloberseiten sind schachbrettartig schwarz oder dunkelbraun und weiß gefleckt.
- Die Flügelunterseiten sind überwiegend weiß bis hellbräunlich gefärbt und haben mehrere graue Flecken, deren Rand etwas dunkler gefärbt ist.
- Das Schachbrett ist in Mitteleuropa häufig. Es lebt in wenig feuchten, grasbewachsenen Gegenden, wie beispielsweise auf Wiesen und Lichtungen und an Straßenrändern und Böschungen, bevorzugt mit kalkigem Boden.
- Das Schachbrettweibchen lässt seine Eier im Flug auf den Boden fallen, wo dann die Raupen schlüpfen.

Quellen:
* https://de.wikipedia.org/wiki/Schachbrett_(Schmetterling)
* https://www.plantura.garden/gruenes-leben/schmetterlinge-die-10-beliebtesten-heimischen-schmetterlingsarten

Optimal, Gaussian, Naïve Bayes Classifiers

1. Classification
2. Optimal Classifier
3. (Optimal) Bayes Classifier
4. Naïve Bayes Classifier

Preliminaries

- Classification
  - Input: d-dimensional feature vector $\boldsymbol{c}$ (computed from pattern $\boldsymbol{f}$), number of classes k
  - **Classifier** maps $\boldsymbol{c}$ to class Ω_κ, $\kappa = 1, \ldots, k$
  - Patterns can be rejected: **Rejection class** Ω_0
- **Many** Classifiers have been proposed!
- Criteria for Classifier Selection
  - Error rate or mean costs/risk of decision quality
  - Computation time for training
  - Computation time for classification
  - Generalization/Robustness
    - Behavior for feature vectors that were not contained in the training set

Optimal, Gaussian, Naïve Bayes Classifiers

1. Classification
2. Optimal Classifier
3. (Optimal) Bayes Classifier
4. Naïve Bayes Classifier

Costs

- A classifier makes erroneous decisions ➔ Costs are assigned to each decision
  - These are application specific → domain expert needed
  - They are not necessarily costs in the sense of currency units
  - Costs allow for **weighing** decisions

- Notation:
  Costs that arise when a feature vector actually belonging to class Ω_κ is classified as Ω_λ

  $r_{\lambda\kappa}$, $\lambda = 0, 1, \ldots, k,$ $\kappa = 1, \ldots, k$

  lambda: what it was classified as
  k: what class it really belongs to

  Required:

  $$0 \leq r_{\kappa\kappa} < r_{0\kappa} < r_{\lambda\kappa} \, , \lambda \neq \kappa$$

  a correct classification should be cheaper than a rejection should be cheaper than a wrong classification

- Costs must be realistic: e.g., if costs for misclassification are too high, the optimal (cheapest) decision is to reject all feature vectors

Optimal Classifier

- A decision for a certain class results in costs
- Based on many decisions, the mean costs (the **risk**) can be estimated

Definition Optimal Classifier:

Minimizes mean costs (the risk) of classification

Optimal Classifier

- The optimal classifier computes a test variable for each class as follows:

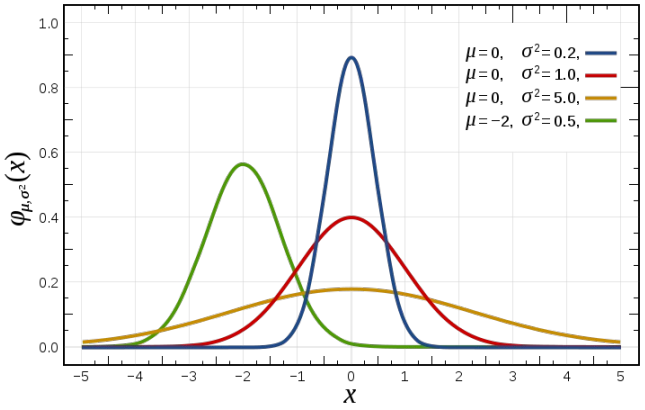
$$u_\lambda(\boldsymbol{c}) = \sum_{\kappa=1}^{k} r_{\lambda\kappa}\, p_\kappa\, p(\boldsymbol{c}|\Omega_\kappa), \qquad \lambda = 0, 1, \ldots, k$$

and decides for the class Ω_κ with **minimal** test variable u_κ

r is costs of classificationc feature vectorp (|k) probability given a c

- Thus, the following probability density functions (PDFs) are required
  - a-priori probability per class (priors): $p_\kappa = p(\Omega_\kappa)$
  - Class-conditional PDF: $p(\boldsymbol{c}|\Omega_\kappa)$

➔ Requires complete statistic information in the form of probability density functions (PDFs)



PDFs for different normal distributions

Details on the derivation of this equation can be found, e.g., in
Niemann, H.: *Klassifikation von Mustern*. 2. überarbeitete Auflage, 2003.

Your Turn !

Exercise 1

Challenges when implementing the Optimal Classifier

probability distribution needed are hard to get and expensive, need large data set with given feature vectors
if feature is missing, we would at least need a large subset that do have that feature or it cannot be evaluated

rejection class is often hard to define in terms of costs (e.g. for domain expert and self); if something isnt classified, i.e. rejected, how much would it cost? hard to answer

Optimal, Gaussian, Naïve Bayes Classifiers

1. Classification
2. Optimal Classifier
3. (Optimal) Bayes Classifier
4. Naïve Bayes Classifier

Bayes Classifier

how to solve previous problems?

- Decision required (rejection not allowed) > no rejection cost which avoids dependency on problem
- Zero-One loss function: $r_{\kappa\kappa} = 0$

  $r_{\lambda\kappa} = 1$ für $\lambda \neq \kappa, \quad \lambda, \kappa = 1, \dots, k$

- Test variable equation then simplifies to

$$u_\lambda(\boldsymbol{c}) = \sum_{\kappa=1,\kappa\neq\lambda}^{k} p_\kappa \, p(\boldsymbol{c}|\Omega_\kappa), \qquad \lambda = 0, 1, \dots, k$$

➔ risk-minimization = minimization of error rate

u1(c): addiere 0,2,3,4,5
u2(c): addiere 1,0,3,4,5
u3(c): addiere 1,2,0,4,5
u4(c): addiere 1,2,3,0,5
u5(c): addiere 1,2,3,4,0

> instead of finding minimum, find maximum of 0 terms

Bayes Classifier

- Decision for class Ω_κ with minimal test variable =
  Decision for class where $p_\kappa \, p(\boldsymbol{c}|\Omega_\kappa)$ is **maximal** which avoids needing sum

- Decision rule: Bayes equation

$$p(\Omega_\lambda|\boldsymbol{c}) = \frac{p_\lambda \, p(\boldsymbol{c}|\Omega_\lambda)}{p(\boldsymbol{c})}$$

- The maximum of the a-posteriori (posterior) probabilities $p(\Omega_\lambda|\boldsymbol{c})$ is the same as for $p_\lambda \, p(\boldsymbol{c}|\Omega_\lambda)$

- $p(\boldsymbol{c})$ is not required for the actual decision
  (but of course, for computing probabilities if desired)

Bayes Classifier Summary

- The Bayes classifier **minimizes** the **error rate**
  - for zero-one loss function,
  - rejection is not allowed
- It computes the k posteriors $p(\Omega_\lambda|\boldsymbol{c})$ and decides for the class with the maximum posterior probability
- There exists ***NO*** classifier having a smaller error rate!
- Any good classifier must approximate the Bayes classifier

Example: k Nearest Neighbor and Bayes

- For very large k and size of sample set $N \rightarrow \infty$
  Error rate of *k*NN approaches that of Bayes classifier
- Estimate of error probability:

$$p_B \leq p_{NN} \leq 2p_B$$

p_B: error probability of Bayes classifier
p_{NN}: error probability of kNN classifier

Assumption: $p_B << 1$

Bayes Classifier – Discussion

- There exists ***NO*** classifier having a smaller error rate!
- Great! So why even consider other classifiers?
- Because: Bayes requires complete statistical information!
  - The PDFs p_λ and $p(\boldsymbol{c}|\Omega_\lambda)$ are usually unknown in actual applications
  - $p(\boldsymbol{c})$ is unknown as well, but can be
    - either computed from $p(\boldsymbol{c}|\Omega_\lambda)$
    - or disregarded as it has no influence on maximization

➔ Need to estimate p_λ and $p(\boldsymbol{c}|\Omega_\lambda)$ from the data!

How to obtain PDFs for Bayes

Goal: Estimate densities p_λ and $p(\boldsymbol{c}|\Omega_\lambda)$

$$p(\Omega_\lambda|\boldsymbol{c}) = \frac{p_\lambda \, p(\boldsymbol{c}|\Omega_\lambda)}{p(\boldsymbol{c})}$$

- Class priors p_λ is a discrete PDF (finite number of classes)
  - Estimate by computing relative frequencies of a (representative) sample dataset ("training data"), or
  - Application specific knowledge, or
  - Assume uniform distribution of classes

i.e. know what the probabilities should be

- Class-conditional PDFs $p(\boldsymbol{c}|\Omega_\lambda)$ could be any PDF
  - Continuous features: assume some distribution,
    - Usually: Gaussian (normal) distribution
  - Discrete (categorical) features:
    - Same as for continuous case, or
    - Same as for priors

relative probabilities

Categorical Optimal Bayes Classifier – Example

- Sample set for training: 1000 customer data records
- Possible classes: customer is buyer (550), customer is non-buyer (450)
- Each record contains the following 5 attributes:
  - Age (<20, 20-30, 30-40, 40-50, 50-60, >60), i.e., 6 different values
  - Income, using 10 different values
  - Number of children (0, 1, 2, 3, 4, 5, more than 5), i.e., 7 different values
  - Country of residence, using 20 countries where our company is active
  - Number of cars (0, 1, 2, 3, 4, more than 4), i.e., 6 different values

$$p(\Omega_\lambda|\boldsymbol{c}) = \frac{p_\lambda \, p(\boldsymbol{c}|\Omega_\lambda)}{p(\boldsymbol{c})}$$

Classifier Training

- Estimate class priors using relative frequencies:
  - p(„buyer“) = 550/1000 = 0.55,
  - p(„non-buyer“) = 450/1000 = 0.45

  ➔ Works well!
- Estimate class-conditional PDFs $p(\boldsymbol{c}|\Omega_\lambda)$
  - Using relative frequencies? Won’t work!
  - Number of components of discrete $p(\boldsymbol{c}|\Omega_\lambda)$: 6×10×7×20×6×2 = 100800
    → considerably more than records in training set

  ➔ impossible to estimate using relative frequencies!

➔ (Optimal) Bayes Classifier can (usually) not be applied to real world problems!

Optimal, Gaussian, Naïve Bayes Classifiers

1. Classification
2. Optimal Classifier
3. (Optimal) Bayes Classifier
4. Naïve Bayes Classifier

How to obtain PDFs for Naïve Bayes – Basic Idea

Goal: Estimate densities p_λ and $p(\boldsymbol{c}|\Omega_\lambda)$

$$p(\Omega_\lambda|\boldsymbol{c}) = \frac{p_\lambda \, p(\boldsymbol{c}|\Omega_\lambda)}{p(\boldsymbol{c})}$$

- Class priors p_λ

➔ Solved

- Class-conditional PDFs $p(\boldsymbol{c}|\Omega_\lambda)$:

➔ Assume independence of features

dont need combination of features (direct multiplication) anymore

Categorical Naïve Bayes Classifier

- Assume that features are statistically independent (uncorrelated)

  can individually estimate probability and mult (prob A and B and C and ...)

  ➔ $p(\boldsymbol{c}|\Omega_\lambda) = \prod_{i=1}^{d} p(c_i|\Omega_\lambda) = p(c_1|\Omega_\lambda) \times p(c_2|\Omega_\lambda) \times \cdots \times p(c_d|\Omega_\lambda)$

➔ Now we can compute variances separately for each feature:

  - Categorical Features: estimate $p(c_i|\Omega_\lambda)$ using relative frequencies from the training data
    ➔ Categorical Naïve Bayes Algorithm

  - Numerical Features: stay tuned

➔ Considerably less data required for training!

Categorical Naïve Bayes – Example Revisited

$$p(\Omega_\lambda|\boldsymbol{c}) = \frac{p_\lambda \, p(\boldsymbol{c}|\Omega_\lambda)}{p(\boldsymbol{c})}$$

- Sample set for training: 1000 customer data records
- Possible classes: customer is buyer (550), customer is non-buyer (450)
- Each record contains the following 5 attributes:
  - Age (<20, 20-30, 30-40, 40-50, 50-60, >60), i.e., 6 different values
  - Income, using 10 different values
  - Number of children (0, 1, 2, 3, 4, 5, more than 5), i.e., 7 different values
  - Country of residence, using 20 countries where our company is active
  - Number of cars (0, 1, 2, 3, 4, more than 4), i.e., 6 different values

Classifier Training

- Estimate class priors using relative frequencies:
  - p(„buyer“) = 550/1000 = 0.55,
  - p(„non-buyer“) = 450/1000 = 0.45

  ➔ Works well!

- Estimate class-conditional PDFs $p(\boldsymbol{c}|\Omega_\lambda)$ assuming independent features using relative frequencies
  - Previously (Optimal Bayes): Number of components of discrete $p(\boldsymbol{c}|\Omega_\lambda)$: (6×10×7×20×6) × 2 = 100800
  - Now (Naïve Bayes): Number of components of discrete $p(\boldsymbol{c} \mid \Omega_\lambda)$: 6×2=12 & 10×2=20 & 7×2==14 & 20×2=40 & 6×2=12
    (in total 98 components, with a maximum of 40 to be estimated at one time)

  ➔ These can (highly likely) be estimated using 1000 data records!

Your Turn !

Exercise 2

Categorical Naïve Bayes Classifier

Relative Frequencies with Zero Value in Categorical Naïve Bayes

- Issue: When using relative frequencies in Categorical Naïve Bayes, all of them must be non-zero, otherwise the total probability will be zero as well:

$$p(\boldsymbol{c}|\Omega_\kappa) = \prod_{i=1}^{n} p(c_i|\Omega_\kappa) = p(c_1|\Omega_\kappa) \times p(c_2|\Omega_\kappa) \times \cdots \times p(c_n|\Omega_\kappa)$$

- Example: sample set containing 1000 records,
  income = low (0), income = average (990), income = high (10)

- Solution: Laplace correction
  - *Simply add 1 to each possible feature value*
    p(income = low) = 1/1003 = 0.001
    p(income = average) = 991/1003 = 0.988
    p(income = high) = 11/1003 = 0.011
  - The „corrected“ relative frequencies are very close to the “uncorrected” ones

Gaussian Naïve Bayes Classifier

- Assume that features are statistically independent (uncorrelated)

  ➔ $p(\boldsymbol{c}|\Omega_\lambda) = \prod_{i=1}^{d} p(c_i|\Omega_\lambda) = p(c_1|\Omega_\lambda) \times p(c_2|\Omega_\lambda) \times \cdots \times p(c_d|\Omega_\lambda)$

➔ Now we can compute variances separately for each feature:

- Categorical Features: estimate $p(c_i|\Omega_\lambda)$ using relative frequencies from the training data
  ➔ Categorical Naïve Bayes Algorithm

- Numerical Features: assume (one-dim) Gaussian distribution, thus $p(c_i|\Omega_\lambda) = \frac{1}{\sqrt{2\pi}\sigma_{i\lambda}} e^{\frac{(c_i-\mu_{i\lambda})^2}{2\sigma_{i\lambda}^2}}$
  and estimate $\mu_{i\lambda}$ and $\sigma_{i\lambda}$ from the $j = 1 \ldots n_\lambda$ training data values $x_{i\lambda j}$
  using $\mu_{i\lambda} = \frac{1}{n_\lambda} \sum_j x_{i\lambda j}$ and $\sigma_{i\lambda}{}^2 = \frac{1}{n_\lambda} \sum_j (x_{i\lambda j} - \mu_{i\lambda})^2$ for each combination of feature i and class λ
  ➔ Gaussian Naïve Bayes Algorithm

  is basically mean value

Computing Classification Probabilities

- How to compute probabilities?

$$p(\Omega_\lambda \mid \boldsymbol{c}) = \frac{p_\lambda p(\boldsymbol{c} \mid \Omega_\lambda)}{p(\boldsymbol{c})}$$

$p(\boldsymbol{c})$ is usually unknown.

- $p(\boldsymbol{c})$ can be computed by **marginalization**:

$$p(\boldsymbol{c}) = \sum_{K=1}^{k} p(\boldsymbol{c}|\Omega_K) p_K$$

sum of priors and PDFs

Your Turn !

Exercise 3

Gaussian Naïve Bayes Classifier

Computing Classification Probabilities - Example

- Example: Categorical Naïve Bayes Classifier Exercise

$$p(\boldsymbol{c}) = p(\boldsymbol{c} \mid \text{buyer} = \text{“yes”}) \times p(\text{buyer} = \text{“yes”}) + p(\boldsymbol{c} \mid \text{buyer} = \text{“no”}) \times p(\text{buyer} = \text{“no”})$$
$$= 0.028 + 0.007 = 0.035$$

Normalizes result to sum = one

$$p(\text{buyer} = \text{“yes”} \mid \boldsymbol{c}) = 0.028 / 0.035 = 0.8$$
$$p(\text{buyer} = \text{“no”} \mid \boldsymbol{c}) = 0.007 / 0.035 = 0.2$$

Naive Bayes spam filtering

- Naive Bayes classifiers are a popular statistical technique of e-mail spam filtering since 1998.
- Advantages
  - Fast.
  - Can be trained on a per-user basis
    - word probabilities are unique to each user
    - can evolve over time with corrective training whenever the filter incorrectly classifies an email.
  - Particularly good in avoiding false positives (where legitimate email is incorrectly classified as spam)
- Approaches to defeat
  - Word Transformations
    - For example, «Viagra» would be replaced with «Viaagra» or «V!agra» in the spam message. The recipient of the message can still read the changed words, but each of these words is met more rarely by the Bayesian filter, which hinders its learning process. As a general rule, this spamming technique does not work very well, because the derived words end up recognized by the filter just like the normal ones.
  - Bayesian poisoning
    - A spammer practicing Bayesian poisoning will send out emails with large amounts of legitimate text (gathered from legitimate news or literary sources). Spammer tactics include insertion of random innocuous words that are not normally associated with spam, thereby decreasing the email's spam score, making it more likely to slip past a Bayesian spam filter. However, with (for example) Paul Graham's scheme only the most significant probabilities are used, so that padding the text out with non-spam-related words does not affect the detection probability significantly.
  - Pictures instead of Text
    - Replace text with pictures, either directly included or linked.

Source: https://en.wikipedia.org/wiki/Naive_Bayes_spam_filtering

Key Takeaways

- Optimal Classifier
  - minimizes mean costs/risk of decision
- (Optimal) Bayes Classifier
  - Decision required (no rejection)
  - (0, 1) cost function
  - Minimizes error rate (optimal with respect to error rate) – no better classifiers regarding this objective exist
  - Any good classifier approximates the Bayes classifier
  - Useless in practice
- Naïve Bayes Classifier
  - Assume statistically independent features
  - Categorical Features: Categorical Naïve Bayes Classifier
    - Estimate using relative frequencies
  - Numerical Features: Gaussian Naïve Bayes Classifier
    - Estimate by assuming class-wise normal distribution of feature vectors
  - There will almost always be dependencies between features, which cannot be modelled by naïve Bayes.
  - In practice, naïve Bayes Classifiers often work very well.